

RaymedTech Digital Platform

Date: February 20, 2026

Prepared by: RaymedTech Tech Team

1. Executive Overview

This document defines the complete and final system blueprint for the RaymedTech Digital Platform (Version 1.0 – MVP Scope Locked).

The objective is to deliver a fully structured, secure, scalable, and rule-driven digital ecosystem.

This document includes:

- Navigation Architecture
- Page-Level Functional Definitions
- Detailed Business Rules
- Workflow Definitions
- Permission Matrix
- System States
- Validation Rules
- Database Logic
- Financial Logic
- Non-Functional Requirements
- Failure Scenarios
- Governance & Compliance

This version is considered implementation-ready and scope-frozen.

2. Platform Scope (Locked MVP)

Included in Version 1.0:

- User Authentication & Role Management
- Union → Entity → Initiative Structure
- Course Management System
- Event Management Engine
- Certificate Engine
- Competition Module (Basic Structured)
- Dashboards (Student, Trainer, Admin)
- Payment Integration
- Notification System
- Admin Control Panel

Excluded from MVP:

- AI Recommendation Engine
- Talent Marketplace
- Advanced Analytics Intelligence
- Public API Access

3. Global Navigation Structure

Top Navigation (Web & Mobile):

- Home
- Courses
- Events
- Union
 - Entities
 - Initiatives
- Competitions
- Community
- About RaymedTech
- Contact
- Login / Sign Up
- Profile (Authenticated Users Only)

Rules:

- Navigation must remain consistent across all pages.
- Profile replaces Login after authentication.
- Mobile uses collapsible navigation drawer.
- Language toggle: Arabic / English.

4. Page-Level Functional Structure

HOME PAGE:

- Hero Section (Title + CTA Buttons)
- Featured Courses (Max 6)
- Upcoming Events (Max 4)
- Active Initiatives Overview
- Platform Statistics
- Footer (Legal + Contact)

COURSE LIST PAGE:

- Filtering (Category, Price, Level)
- Sorting (Newest, Popular)
- Pagination Enabled

COURSE DETAILS PAGE:

- Full Course Metadata
- Lessons List (Locked if not enrolled)
- Reviews
- Enrollment Button
- Validation of Login & Payment

EVENT DETAILS PAGE:

- Event Metadata
- Capacity Display
- Countdown Timer
- Registration Button
- Refund Policy Display

DASHBOARD (Role-Based):

- Student: My Courses, My Events, Certificates, Notifications
- Trainer: My Courses, Revenue, Reviews
- Admin: Users, Approvals, Analytics, Reports

5. Detailed Functional Rules

COURSE RULES:

- Lesson marked complete if $\geq 80\%$ video watched.
- Course completion requires $\geq 80\%$ lessons.
- Final assessment required.
- Passing score $\geq 60\%$.
- Max attempts = 3.
- Certificate auto-generated PDF with unique ID & QR validation.

EVENT RULES:

- Capacity cannot be exceeded.
- Registration disabled when full.
- Refund allowed within 7 days if attendance not marked.
- Status transitions strictly controlled.

COMPETITION RULES:

- Individual or Team mode defined at creation.
- Max team size configurable.
- Judge scoring weighted (Admin configurable).
- Tie-breaker: highest technical score.

6. Workflow Definitions

COURSE WORKFLOW:

Draft → Submitted → Approved → Published → Archived/Suspended

EVENT WORKFLOW:

Draft → Submitted → Approved → Published → Completed/Cancelled

USER WORKFLOW:

Pending Verification → Active → Suspended → Soft Deleted

PAYMENT WORKFLOW:

Initiated → Processing → Successful → Failed → Refunded

7. Permissions Matrix (Summary)

STUDENT:

- Enroll Courses
- Register Events
- Join Initiatives
- Leave Reviews

TRAINER:

- Create/Edit Own Courses
- View Own Revenue
- Submit Course for Approval

ENTITY OWNER:

- Create Initiatives
- Assign Initiative Leaders
- View Entity Reports

ADMIN:

- Approve Courses & Events
- Suspend Users
- Manage Entities
- Configure Revenue Rules

SUPER ADMIN:

- Full System Control
- Role Assignment
- System Configuration

8. Required Validations

USER:

- Unique email required.
- Password minimum 8 characters.
- Email verification mandatory before enrollment.

COURSE:

- Title mandatory.
- Minimum one lesson required before submission.

EVENT:

- Capacity > 0
- Start date < End date

PAYMENT:

- Amount must match product price.
- Unique transaction ID required.

9. Database Logic Overview

Core Tables:

users
roles
user_roles
entities
initiatives
courses
lessons
enrollments
events
event_registrations
payments
certificates
reviews
notifications
audit_logs

All foreign keys enforced.

Soft delete implemented using deleted_at column.

Audit logs required for privileged actions.

10. Business & Revenue Logic

Default Revenue Split:

- 70% Trainer
- 20% Platform
- 10% Entity

Configurable by Admin.

Payout Rules:

- Monthly payout cycle.
- 14-day refund buffer.
- Minimum payout threshold required.

Refund Conditions:

- Within 7 days.
- Course progress < 30%.

11. Non-Functional Requirements

Performance:

- Page load < 3 seconds.
- API response < 800ms.

Security:

- JWT Authentication.
- HTTPS mandatory.
- Role-Based Access Control.
- Password hashing (bcrypt).
- Rate limiting.

Scalability:

- Cloud-native architecture.
- CDN required for media.
- Horizontal scaling supported.

Availability:

- 99.9% uptime target.
- Daily automated backups.
- 30-day retention.

12. Failure & Edge Case Handling

- Prevent duplicate enrollment.
- Prevent overbooking via transaction control.
- Handle payment failures gracefully.
- Immediate access revocation if user suspended.
- Admin rejection must include reason and notification.

13. Legal & Compliance

- Public Privacy Policy required.
- Terms of Service mandatory acceptance.
- Organizational verification required before activation.
- Financial compliance enforced.
- Intellectual property protection embedded in system rules.

14. Implementation Gaps & Required Technical Specifications

(Mandatory Completion Before Development Execution)

This section defines all missing technical specifications required to ensure full implementation readiness and eliminate ambiguity during development.

All items listed below are mandatory for engineering completion.

14.1 API Contract Specification (Critical Requirement)

All backend endpoints must be formally defined.

Each endpoint must include:

- Endpoint URL
- HTTP Method (GET, POST, PUT, DELETE)
- Authentication requirement
- Request body schema
- Response body schema
- Error response schema
- Status codes

Example format:

COURSE CREATION API

Endpoint: POST /api/courses

Request:

```
{
  "title": "Course Title",
  "description": "Course Description",
  "price": 100,
  "entity_id": 5
}
```

Response:

```
{
  "id": 101,
  "status": "draft",
  "created_at": "timestamp"
}
```

Error Example:

```
{
  "error": "VALIDATION_ERROR",
  "message": "Title is required"
}
```

This specification must be completed for all modules.

14.2 State Transition Matrix (Strict Enforcement Required)

All system objects must define explicitly allowed and forbidden state transitions.

Example: Course State Transition Matrix

Allowed transitions:

Draft → Submitted
Submitted → Approved
Approved → Published
Published → Archived
Published → Suspended

Forbidden transitions:

Published → Draft
Archived → Published
Suspended → Draft

All state transitions must be validated at backend level.

Unauthorized transitions must be rejected.

14.3 Payment Integration Technical Specification

The payment system must define:

- Payment gateway provider
- Payment initiation endpoint
- Payment verification method
- Webhook handling logic
- Payment confirmation trigger logic
- Retry logic for failed payments

Enrollment access must be granted ONLY after payment status = Successful.

Payment status must be verified using gateway callback or webhook.

Manual status manipulation must be restricted to Admin only.

14.4 Enrollment Rules & Access Control Specification

Enrollment rules must explicitly define:

- User cannot enroll twice in the same course
- User cannot enroll in archived or suspended courses
- Enrollment must be linked to verified payment
- Enrollment must be revoked automatically upon refund approval
- Enrollment must not bypass payment validation

System must enforce enrollment uniqueness constraint.

14.5 Certificate Generation Trigger Logic

Certificate generation must occur ONLY when ALL conditions are met:

- Course completion $\geq 80\%$ lessons
- Final assessment passed
- Passing score $\geq$ required threshold
- Enrollment status active
- Course status Published or Completed

Certificate must include:

- Unique certificate ID
- User ID reference
- Course ID reference
- Issue timestamp
- QR validation link

Certificates must be non-editable and permanently stored.

14.6 Detailed Role-Permission Matrix (Mandatory Expansion)

A complete permission matrix must define ALL actions for ALL roles.

Example actions include:

- Create course
- Edit course
- Delete course
- Publish course
- Suspend course
- View users
- View financial data
- Issue refunds
- Manage entities

Permissions must be enforced at backend level.

Frontend restrictions alone are not sufficient.

14.7 Database Schema Definition (Column-Level Required)

Each table must include full schema definition:

Example:

COURSES TABLE

Required columns:

- id (Primary Key)
- title
- description
- price
- status
- entity_id (Foreign Key)
- created_by
- created_at
- updated_at
- deleted_at

All foreign keys must be explicitly defined.

All constraints must be documented.

14.8 Error Handling & Failure Recovery Logic

System must define structured error handling for:

- Payment failures
- API failures
- Database failures
- Validation failures
- Authorization failures

Error responses must follow unified format:

```
{  
  "error_code": "ERROR_IDENTIFIER",
```

```
"message": "Human readable message",  
"timestamp": "timestamp"  
}
```

System must prevent partial or inconsistent state creation.

14.9 Concurrency & Transaction Safety Rules

System must prevent race conditions and duplicate transactions.

Required protections:

- Database transaction locking for enrollment
- Database transaction locking for event registration
- Atomic payment processing
- Prevention of duplicate certificate generation

All critical operations must use database transactions.

14.10 Notification Trigger Matrix

All system-triggered notifications must be explicitly defined.

Example triggers:

Course Approved → Notify Trainer
Payment Successful → Notify Student
Course Completed → Notify Student
Certificate Issued → Notify Student
Refund Processed → Notify User
Account Suspended → Notify User

Each notification must define:

- Trigger event
 - Recipient role
 - Notification type (Email, In-app, Push)
-

14.11 Media Storage & Delivery Architecture

All media files must use secure, scalable storage.

Specification must define:

- Storage provider (Cloud Storage Recommended)
- File access control mechanism
- Signed URL or secure streaming method
- CDN integration for performance

Direct public access to raw storage must be restricted.

14.12 Authentication & Session Security Specification

Authentication system must define:

- JWT expiration duration
- Refresh token mechanism
- Token revocation logic
- Session invalidation on logout
- Session invalidation on password change

Unauthorized access must be strictly prevented.

14.13 Deployment & Infrastructure Architecture

System deployment specification must define:

- Hosting environment (Cloud provider recommended)
- Containerization strategy (Docker recommended)
- Database hosting configuration
- Backup and recovery procedures
- Environment separation (Development, Staging, Production)

Deployment process must be fully documented.

14.14 Audit Logging Requirements

All critical system actions must be logged.

Examples include:

- Login attempts
- Payment actions
- Course approval/rejection
- Permission changes
- Refund processing

Audit logs must include:

- User ID
- Action type
- Timestamp
- IP address

Logs must be immutable.